

C++ for Quant Interviews: The Complete Guide

From RAII to Low-Latency Systems
Correctness, Memory, and Performance Made Practical

Amit Kumar Jha

January 24, 2026

Contents

How to Read This Guide	3
Scope and Disclaimer	3
Quick Start: A Minimal C++ Quant Setup	3
Boundary Map: Where C++ Adds Value on a Desk	3
Module 0: C++ Fundamentals Refresher	5
Part I — C++ Foundations (Syntax as Financial Logic)	8
Module 1: Types as Financial Objects	8
Part II — The Engine Room (How C++ Actually Runs)	11
Module 2: The Machine Model (Stack, Heap, Lifetime, UB)	11
Module 3: RAII, Ownership, and Exception Safety	13
Module 4: STL Containers for Trading	15
Module 5: Algorithms and “Think in Transformations”	17
Module 6: Numerical Precision for Finance	19
Module 7: Performance Engineering	21
Module 8: Concurrency and Parallelism	23
Module 9: Templates (Practical, Not Metaprogramming)	25
Module 10: Build Systems and Tooling (CMake, Flags, Sanitizers)	27
Module 11: Testing and Benchmarking Quant Code	29
Part III — The Quant System Toolkit (From Functions to Architecture)	30
Module 12: C++ and Python Interop (Where the Boundary Lives)	30
Module 13: System Design Patterns in Quant C++	31

Practitioner Insight: Vector growth diagram

Most implementations grow capacity geometrically (often 2x). Reallocation copies/moves all elements, which can dominate latency if done in hot loops. If you know the approximate size, reserve.

4.4 Portfolio with metadata: “map to a struct”

Listing 13: Portfolio where each position stores multiple risk fields

```
1 #include <unordered_map>
2 #include <string>
3
4 struct Position {
5     long long qty{};
6     double delta{};
7     double vega{};
8 };
9
10 using RiskBook = std::unordered_map<std::string, Position>;
```

The 3-Second Answer: Container choice

“For market data arrays and time series I default to vector because it’s contiguous. For a portfolio keyed by symbol I use unordered_map for O(1) lookup. I avoid list unless I truly need node splicing.”

Cheat Sheet: Module 4

- Default to contiguous storage: vector, array.
- Use deque for FIFO/rolling windows; avoid pop(0) patterns.
- Hash maps for lookup; ordered maps for sorted keys and range queries.
- In performance work, allocations are often the true bottleneck: reserve.

Module 19: Time-Pressure Techniques and Code Review Red Flags

Module Overview

- Time pressure: start correct, then optimize. Communicate assumptions explicitly.
- Code review red flags: hidden ownership, unnecessary `shared_ptr`, unbounded allocations.
- Interview execution: talk in invariants, not syntax.

3-Second Interview Answer

Under pressure I write the simplest correct solution, state complexity, then improve locality / allocations if asked.

Memory Hook

"Correct → clear → fast" (in that order).

19.1 The 5-step time-pressure protocol

1. Clarify inputs and edge cases (30s).
2. State approach and complexity (30s).
3. Write skeleton (1 min).
4. Fill core logic.
5. Test verbally with small cases.

19.2 Code review red flags (C++ edition)

- Raw `new/delete` without strong reason.
- Missing `const` and unclear ownership.
- Returning references to locals.
- Vector growth without `reserve` in known-size loops.
- Silent narrowing conversions (e.g., `double -> int`).
- Exception-unsafe code that leaks resources.

19.3 What impresses reviewers

- Clear types (`int64_t` for quantities), named constants (avoid magic 252 without comment).
- RAII everywhere: files, locks, memory.
- Tests with float tolerances and invariants.
- Build instructions and sanitizer targets.

Cheat Sheet: Module 19

- Process beats panic: clarify, skeleton, then logic.
- In C++, ownership clarity is the main senior signal.
- Optimize only after correctness and measurement.

Appendix I: Container Complexity and Invalidation (Safe Rules)

This table is intentionally conservative. In low-level C++, the fastest way to ship a bug is to assume an iterator remains valid after a container mutation. When in doubt: reacquire iterators after modifications, and use `reserve` to control rehash/reallocation.

Container	Typical lookup	Invalidation rule of thumb (safe)
<code>vector</code>	index $O(1)$	Reallocation invalidates <i>all</i> pointers/refs/iters. Insert/erase invalidates at-and-after position.
<code>deque</code>	index $O(1)$	Treat insert/erase/push/pop as invalidating iterators; for portable code, do not store iterators across mutations.
<code>list</code>	search $O(n)$	Insert does not invalidate existing iterators/refs; erase invalidates only erased element. Good for stable node addresses, poor for locality.
<code>map</code>	$O(\log n)$	Insert does not invalidate existing iterators/refs; erase invalidates only erased element.
<code>unordered_map</code>	avg $O(1)$	Rehash invalidates <i>all</i> iterators/refs/pointers. Inserts may trigger rehash; call <code>reserve</code> when possible. Erase invalidates erased only (unless rehash).

Practitioner Insight: How to state this in interviews

“For vectors, I assume any growth can invalidate all references and iterators. For `unordered_map`, I remember that rehash invalidates everything, so I reserve upfront if the size is known. If I need stable references, I choose a container that guarantees it, or I store indices/IDs instead of iterators.”